CheckIt.AI — Pipeline de transformation et schéma de données

Étape 03 — Transformer, contrôler et préparer le chargement
État du projet vérifié le 7 septembre 2026

1. Objectif et grain des données

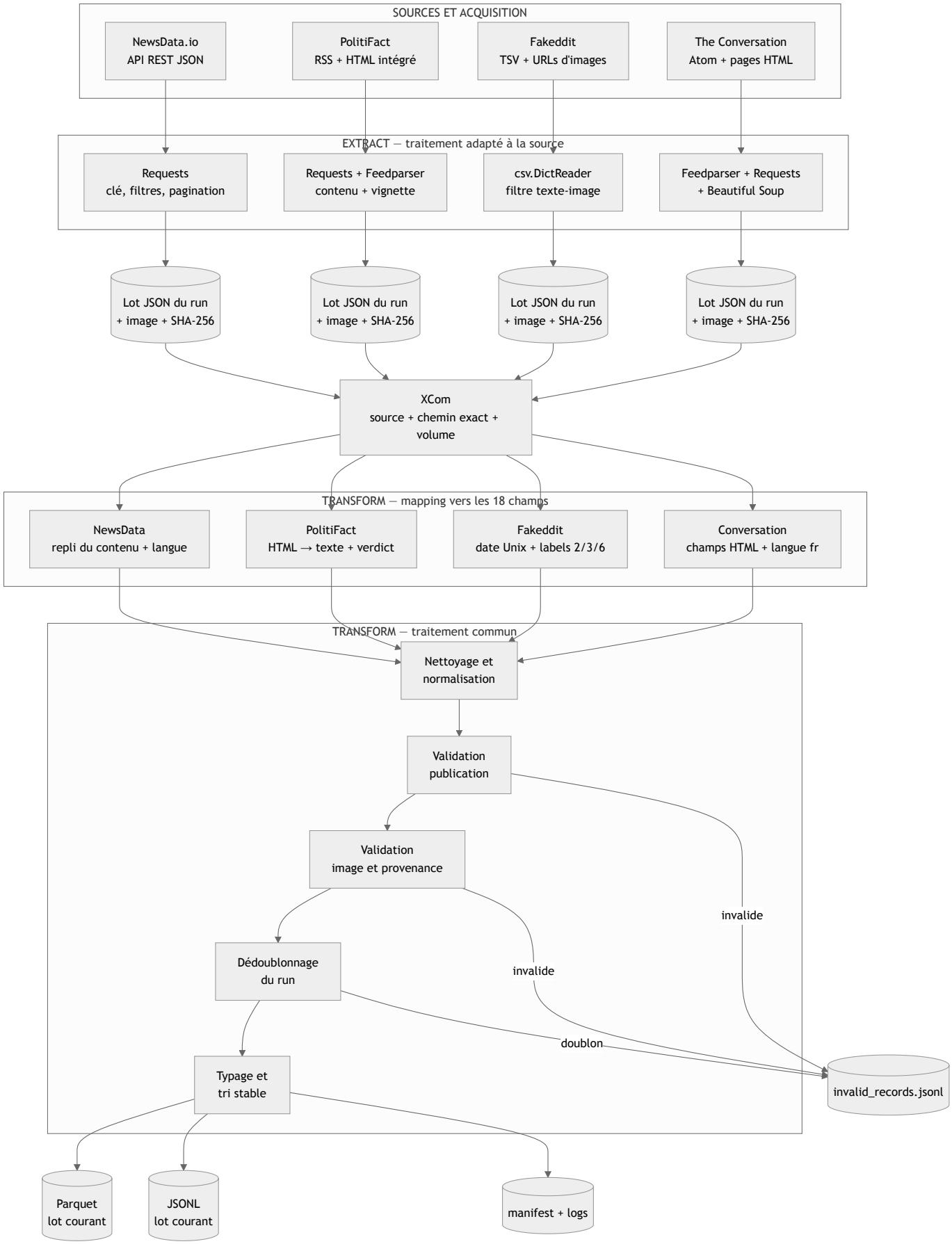
Cette étape transforme uniquement les quatre lots bruts produits par le run courant en un contrat commun, reproductible et journalisé. Airflow transmet leurs chemins exacts par XCom ; `data/processed/publications.parquet` représente donc l'incrément à charger dans PostgreSQL.

Son grain est : **une ligne par publication unique dans le run, associée à exactement une image locale validée**.

- **clé primaire logique :** `publication_id` ;
- **nombre de colonnes :** 18 ;
- **image :** obligatoire et stockée sur la même ligne que son texte ;
- **label :** facultatif, conservé sans réinterprétation ;
- **dédoublonnage secondaire :** `source_url + image_sha256`.

Le schéma conceptuel reprend volontairement cette structure plate. Il correspond à la table PostgreSQL réellement alimentée par le pipeline, sans ajouter d'entités inutiles.

2. Pipeline de l’acquisition à la transformation



3. Traitement propre à chaque source

Source	Acquisition	Mapping vers le contrat commun
NewsData.io	API REST avec `Requests`, clé d'environnement, filtres, pagination et retries	`article_id` devient `newsdata_<id>` ; le texte suit le repli `content → description → ai_summary` ; `french` devient `fr`
PolitiFact	RSS téléchargé avec `Requests`, puis lu avec `Feedparser`	SHA-256 de l'ID RSS ou de l'URL sur 16 caractères ; `BeautifulSoup` nettoie `content_html` et lit le verdict du `flat-meter`
Fakeddit	TSV lu ligne par ligne avec `csv.DictReader` ; un curseur persistant reprend après le lot réussi précédent	ID Reddit préfixé par `fakeddit_` ; date Unix vers UTC ; labels 2/3/6 regroupés dans une chaîne JSON
The Conversation France	Flux Atom pour les URLs, puis page téléchargée et analysée avec `Beautiful Soup`	ID numérique de l'article, ou hash stable de l'URL ; texte HTML déjà extrait ; langue fixée à `fr` ; aucun label inventé

Les quatre fonctions `transform_<source>()` produisent le même dictionnaire provisoire. Le traitement commun peut alors appliquer les mêmes validations à toutes les sources.

4. Traitement commun, dédoublonnage et journalisation

Le script `scripts/transform_data.py` suit quatre étapes visibles dans `logs/transform.log` :

1. **Lecture :** contrôle des seuls chemins fournis avec `--input-files` ou reçus par XCom, puis vérification de chaque enveloppe JSON.
2. **Mapping et validation :** nettoyage Unicode et espaces, dates UTC, domaine, langue, URLs, labels et preuves d'image.
3. **Dédoublonnage du run :** conservation de la première ou de la dernière occurrence selon `--duplicate-policy`.
4. **Export :** typage avec pandas, tri stable, écriture atomique et création du manifeste.

Ce déroulé est découpé en fonctions ciblées : `map_and_validate_record()` traite une publication, `transform_batch()` isole un lot, `transform_inputs()` consolide les lots et `run_transformation()` pilote les sorties. Cette séparation permet de tester ou de faire évoluer une règle sans dupliquer le pipeline.

Dans les lots du run courant, un doublon est détecté par :

- la même valeur de `publication_id` ; ou
- le même couple `source_url + image_sha256`.

Chaque rejet contient la source, le fichier d'entrée, l'identifiant brut et la raison. Le manifeste enregistre les paramètres, les métriques, les hash des entrées et sorties, la version du schéma et le hash du pipeline.

Paramètres principaux :

Paramètre	Rôle
`--input-files <chemins>`	Fournir exactement un lot courant pour chacune des quatre sources
`--duplicate-policy keep-first/keep-latest`	Choisir l'occurrence conservée
`--output-format parquet/jsonl/both`	Choisir les fichiers produits

5. Contrat final de 18 colonnes

Champ	Type final	Obligatoire	Rôle
`publication_id`	chaîne	Oui	Identifiant stable et PK logique
`source_name`	chaîne	Oui	Source d'acquisition
`source_domain`	chaîne	Oui	Domaine éditorial ou cible
`source_url`	URL HTTP(S)	Oui	Traçabilité de la publication
`title`	chaîne	Conditionnel	Texte court ; titre ou texte obligatoire
`text`	texte long	Conditionnel	Contenu destiné au NLP

Champ	Type final	Obligatoire	Rôle
`image_url`	URL HTTP(S)	Oui	URL distante de l'image liée
`image_path`	chemin	Oui	Image locale validée
`image_size_bytes`	entier	Oui	Contrôle du fichier vide ou modifié
`image_sha256`	chaîne hexadécimale	Oui	Intégrité et dédoublonnage
`image_provenance_status`	chaîne	Oui	État de la preuve URL-image
`published_at`	date-heure UTC	Oui	Date de publication
`language`	ISO 639-1	Oui	`fr`, `en`, etc.
`author`	chaîne	Non	Métadonnée éditoriale
`source_label_raw`	chaîne	Non	Label original non converti
`source_label_scheme`	chaîne	Non	Système donnant son sens au label
`label_provenance`	chaîne	Non	Auteur ou méthode de l'annotation
`collected_at`	date-heure UTC	Oui	Date de collecte du lot

Génération de `publication_id`

Source	Règle
NewsData.io	`newsdata_` + `article_id` fourni par l'API
PolitiFact	`politifact_` + 16 premiers caractères du SHA-256 de l'ID RSS ou de l'URL
Fakeddit	`fakeddit_` + ID Reddit fourni dans le TSV
The Conversation	`theconversation_` + ID numérique de l'article ; à défaut, 16 caractères du SHA-256 de l'URL

Le préfixe évite les collisions entre deux sources qui utiliseraient le même identifiant natif.

6. Schéma conceptuel pour la phase Load

Le contrat final contient une seule entité. Chaque ligne regroupe une publication, son texte, son image, ses métadonnées et son éventuel label. Le manifeste de transformation conserve séparément les informations d'exécution du pipeline.

PUBLICATION_MULTIMODALE		
string	publication_id	PK
string	source_name	
string	source_domain	
string	source_url	
string	title	
text	text	
string	image_url	
string	image_path	
integer	image_size_bytes	
string	image_sha256	
string	image_provenance_status	
datetime	published_at	
string	language	
string	author	
string	source_label_raw	
string	source_label_scheme	
string	label_provenance	
datetime	collected_at	

Lecture du schéma

`PUBLICATION_MULTIMODALE` représente une publication complète et directement exploitable :

- `publication_id` est la **PK** et identifie la ligne ;
- `title` et `text` représentent la partie textuelle ;
- les cinq champs `image_*` représentent l'image associée et sa preuve d'intégrité ;
- les champs `source_*`, `author`, `published_at`, `language` et `collected_at` assurent la traçabilité ;
- les trois champs de label restent facultatifs et indissociables.

Aucune **FK** n'est nécessaire puisqu'il n'existe qu'une table. Le lien texte-image est garanti par leur présence sur la même ligne et par les contrôles effectués pendant Transform.

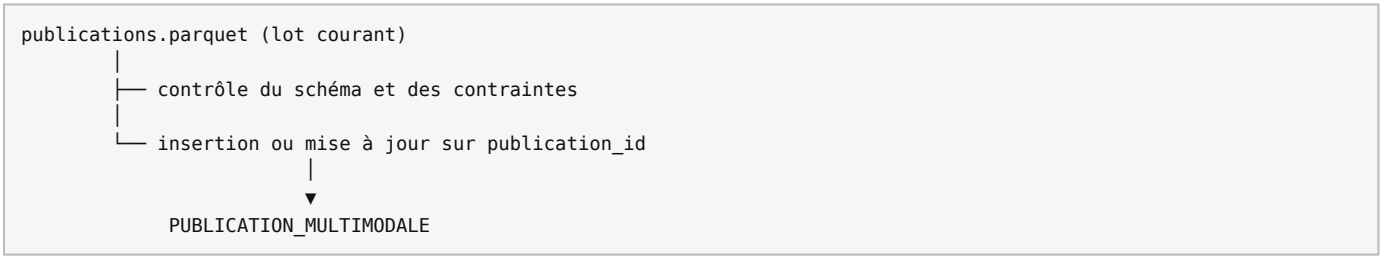
7. Contraintes et chargement

Contraintes d'intégrité

- `publication_id` est unique et non nul ;
- `source_name` et `source_domain` sont non nuls ;
- `title` ou `text` doit être renseigné ;
- `source_url` et `image_url` sont des URLs HTTP(S) ;
- `image_path` désigne un fichier local validé ;
- `image_size_bytes > 0` ;
- `image_sha256` contient 64 caractères hexadécimaux ;
- `image_provenance_status` indique comment l'association URL-image a été vérifiée ;
- `published_at` et `collected_at` sont des dates UTC ;
- `language` contient deux lettres minuscules ;
- les trois champs de label sont tous présents ou tous absents ;
- le couple `source_url + image_sha256` est unique après dédoublonnage.

Chargement réalisé

Le DAG charge le Parquet incrémental dans la table cumulative `PUBLICATION_MULTIMODALE`, qui possède les mêmes 18 colonnes :



L'upsert utilise la PK `publication_id` : une publication déjà connue est mise à jour, une nouvelle est insérée. Le manifeste reste séparé comme fichier d'audit du run ; il n'est pas mélangé aux publications.

8. Lecture des résultats du pipeline

Le manifeste `data/processed/transformation_manifest.json` décrit uniquement les quatre lots reçus par XCom pendant le run courant :

- 4 lots exacts et 60 occurrences lus ;
- 60 publications exportées : 20 Fakeddit, 20 PolitiFact, 10 The Conversation et 10 NewsData.io ;
- 0 doublon dans le run ;
- 0 rejet de validation ;
- schéma `1.3` et 18 colonnes.

Le Parquet et le JSONL représentent ce seul incrément. Le collecteur KPI vérifie le mode `incremental_exact_batches`, la version de schéma `1.3`, la présence des quatre sources et la correspondance entre le manifeste et le Parquet. PostgreSQL constitue l'état cumulatif au moyen de l'upsert. Hors Airflow, une exécution équivalente doit fournir les quatre chemins exacts à l'option obligatoire `--input-files`.